CPSC 481
Artificial Intelligence

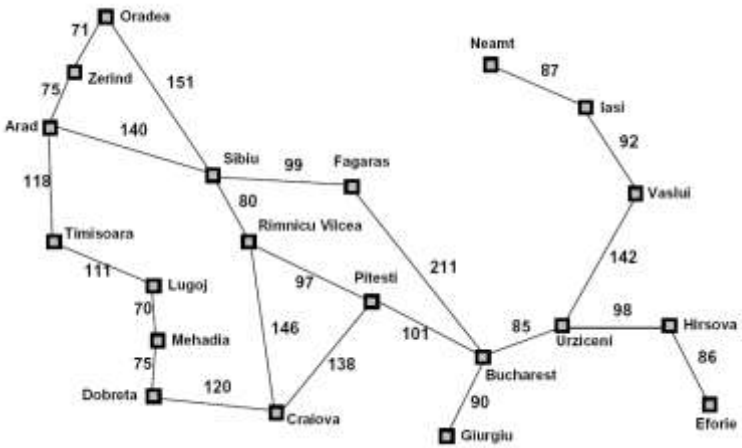
Dr. Anand Panangadan
apanangadan@fullerton.edu

What we will cover today

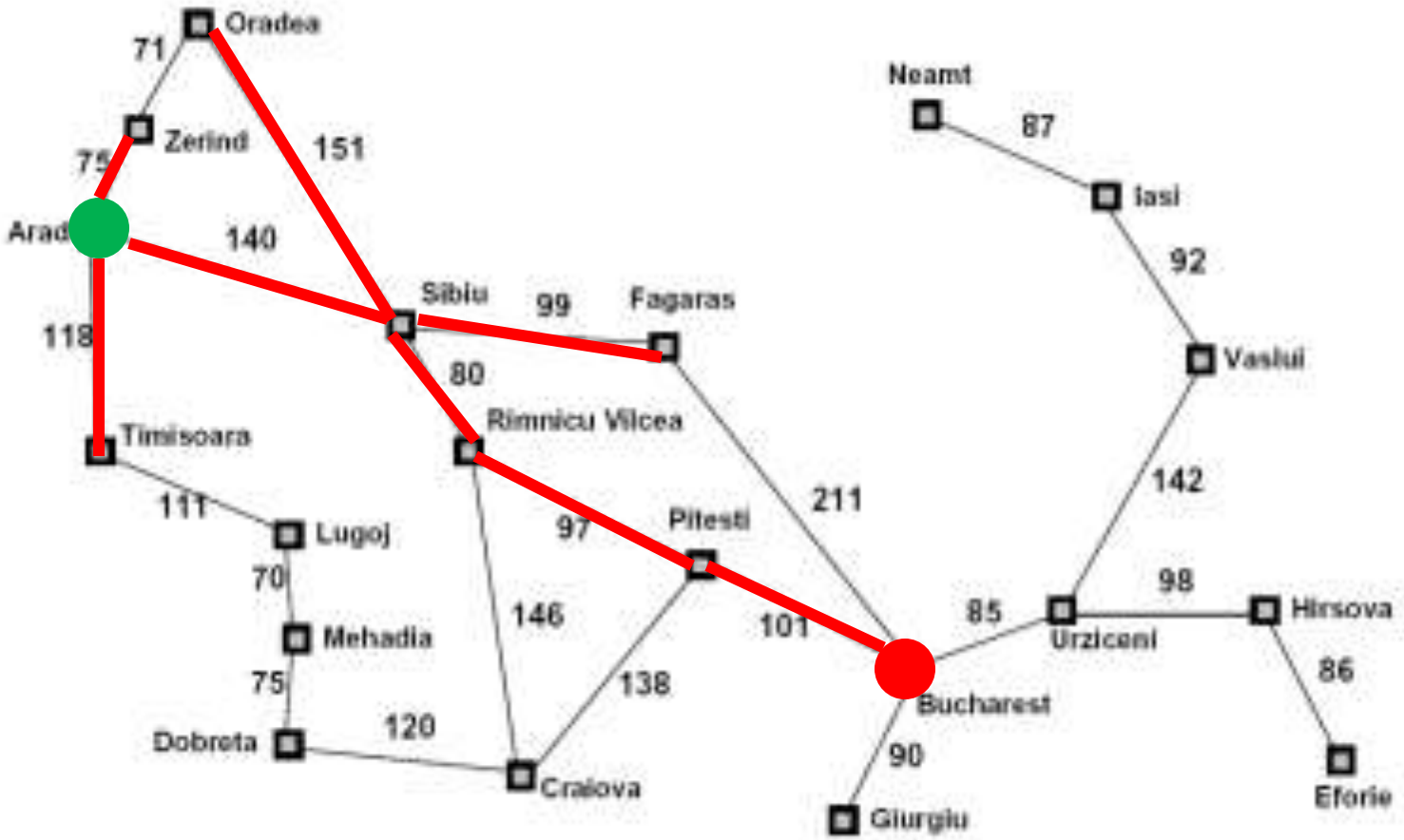
- Optimization problems and Local search algorithms
  - Hill climbing search
  - Simulated annealing
  - Genetic algorithms

Path vs. Goal

- BFS/DFS/UCS/A* all return a path to a goal state
  - The goal state is known (consider sliding tile puzzle)
  - Difficult part is getting there from the initial state
- Good:
  - Optimal
    - Systematic search of the entire state space
- Bad:
  - Slow
  - high/infeasible memory requirement
- Applications
  - Solving puzzles
  - Robot path planning
  - Parsing natural language (text -> grammar)
  - Wireless network routing



Path vs. Goal


Oradea
71
Zerind
151
75
Arad
140
118
Sibiu
99
Fagaras
80
Timisoara
Rimnicu Vilcea
111
Lugoj
97
Pitesti
211
70
Mehadia
146
101
75
138
Dobreta
120
Craiova
Bucharest
90
Giurgiu
Neamt
87
Iasi
92
Vaslui
142
85
Urziceni
98
Hirsova
86
Eforie

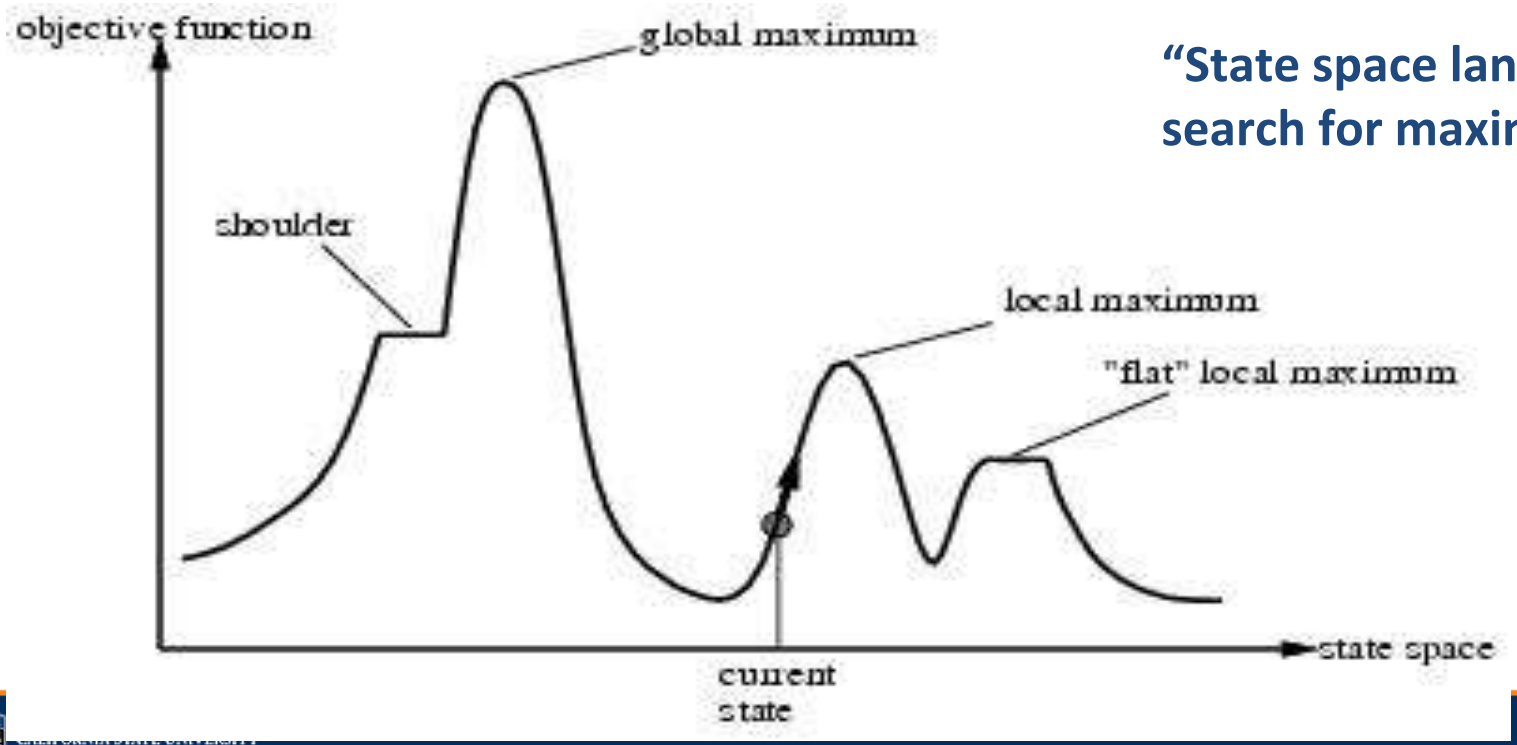
Path vs. Goal

- Optimization problems
  - Find any goal state
  - Path used to find the goal state is not important
- Applications
  - Course scheduling: fit as many classes in classrooms
  - Job scheduling
  - Constraint satisfaction problems: Sudoku
- Good:
  - Don’t worry about keeping track of paths
  - Fast
- Bad:
  - Lose optimality

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

Optimization problems

- Find the "**best**" state according to an objective function
  - Objective function: state → number
- Find largest value of objective function: *maximization*
- Find smallest value of objective function: *minimization*



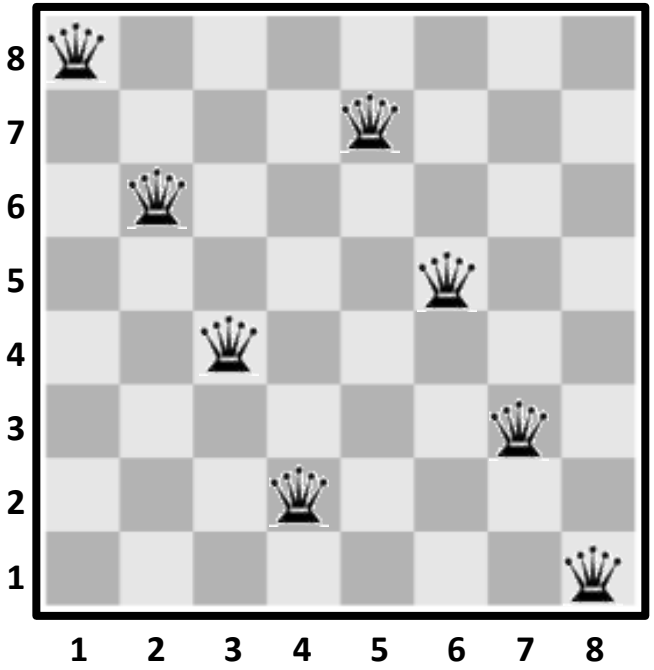
"State space landscape" for search for maximum

Local search

- Challenge:
  - Very large number of states
- Key idea:
  - Keep track of a "current state"
    - And its objective function value
  - Consider only successors of current state
    - Successor of a state: a state that is only "slightly different" from that state (also called neighbor)
    - If a successor is better than current state (higher objective function value), replace current state with this successor
  - Ignore paths
  - Repeat

Example optimization problem

- 8-queens problem
  - Place 8 queens on a chessboard such that no queen attacks another
- Size of state space?
- How to represent the state?
  - ((1,8),(2,6),(3,4),(4,2), …)
  - (8, 6, 4, 2, 7, 5, 3, 1) simpler
- Successors?
  - All states where only **one queen** is moved
- Objective function
  - Minimization: Number of pairs of queens that are attacking each other
    - For shown state?
    - Global minimum: 0
    - Is this a local minimum?
  - Maximization: Number of pairs of queens that *do not attack* each other
    - Is this a global maximum?

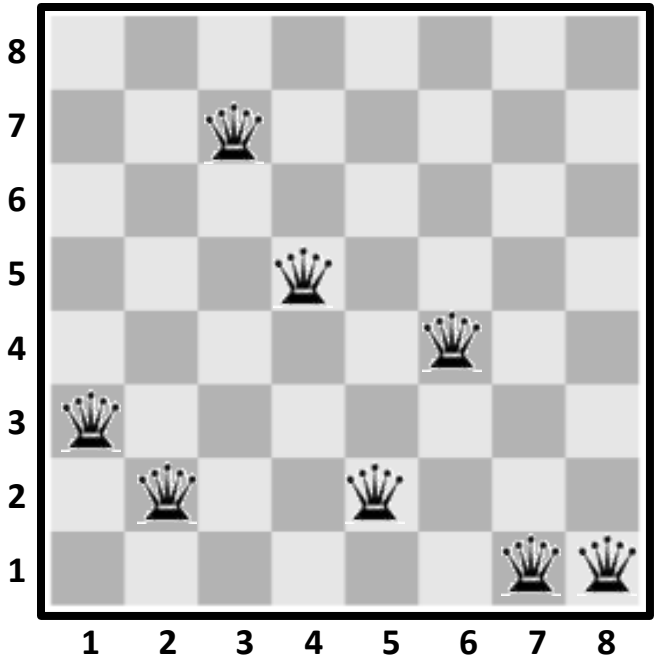


Global and local maximum

- Global maximum:
  - A state that has the highest possible objective function value in the entire state space
- Local maximum
  - A state that has the highest possible objective function value *among all its successors*
- Similarly for global and local minimum

Classwork

- Represent this board as an 8-tuple
- Calculate the objective function
  - number of attacking pairs
- Generate a few successor states
  - List the 8-tuples
- Calculate the objective function for a few successor states



Local search

HILL CLIMBING

Hill-climbing search

```
function HILL-CLIMBING( problem) return a state that is a local maximum
    input: problem, a problem
    local variables: current, a node.
                     neighbor, a node.

    current ← MAKE-NODE(INITIAL-STATE[problem])
    loop do
        neighbor ← a highest valued successor of current
        if VALUE [neighbor] ≤ VALUE[current] then return STATE[current]
        current ← neighbor
```

Algorithm sketch

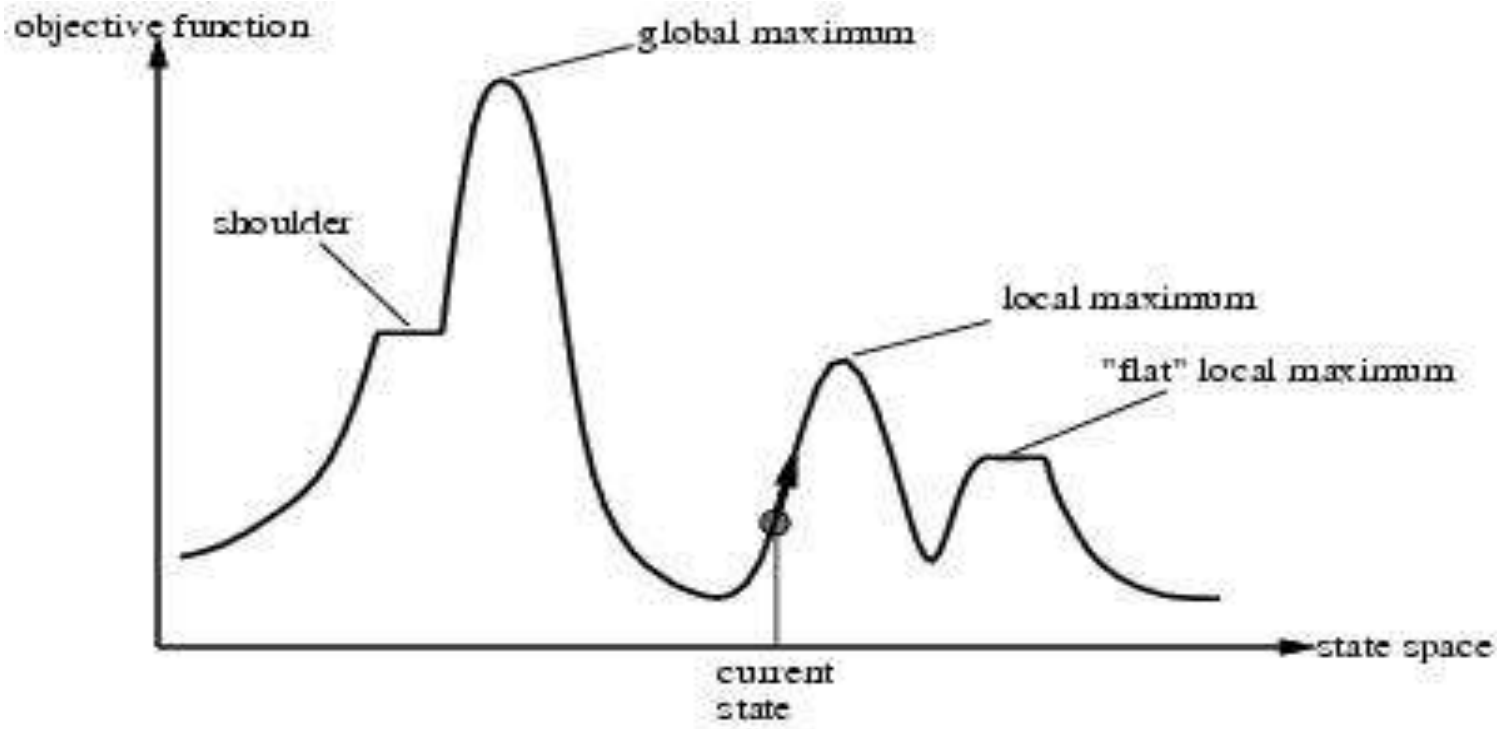
1. Expand the current state of the search.
2. Evaluate its successor states using the objective function.
3. Select the **best successor state** for further expansion.

Continue steps 1 - 3 until no better state is found

Hill-climbing search

- **Advantages:**
  - Uses very little memory
  - Can often find reasonable solutions in large or infinite (continuous) state spaces.
- But not optimal
  - Gets stuck in local maxima
  - Greedy local search

Problems of Hill-climbing



Local maxima
gets stuck when it reaches a state that has no neighboring states that are better.
Shoulder
all direct successors are not better; but a close neighbor is
Plateau
All neighboring states are not better; how far away should I keep looking for a better state?

Variations of Hill-climbing

- **Steepest ascent**
  - Version seen previously
- **Stochastic hill-climbing**
  - Of all better successors, choose one randomly
  - Intentionally not always choosing the best
  - Slower but finds better solutions in some cases
- **First-choice hill-climbing**
  - Randomly choose first better successor
  - Suitable when there are thousands of successor states
- **Random-restart hill climbing**
  - If you get stuck, start over from a random state (but note the best state seen so far)
- **Beam search**
  - Maintain a pool of *k* current states – the "beam"
  - Generate all successors of the *k* current states; keep only the best *k successors* for the new beam

Local search

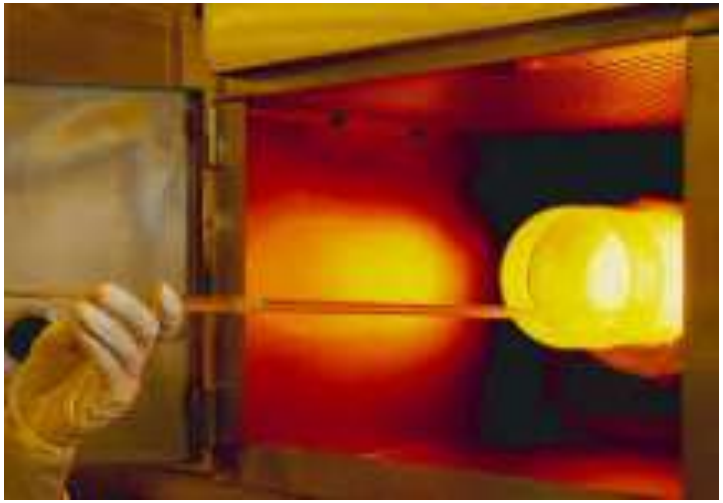
SIMULATED ANNEALING

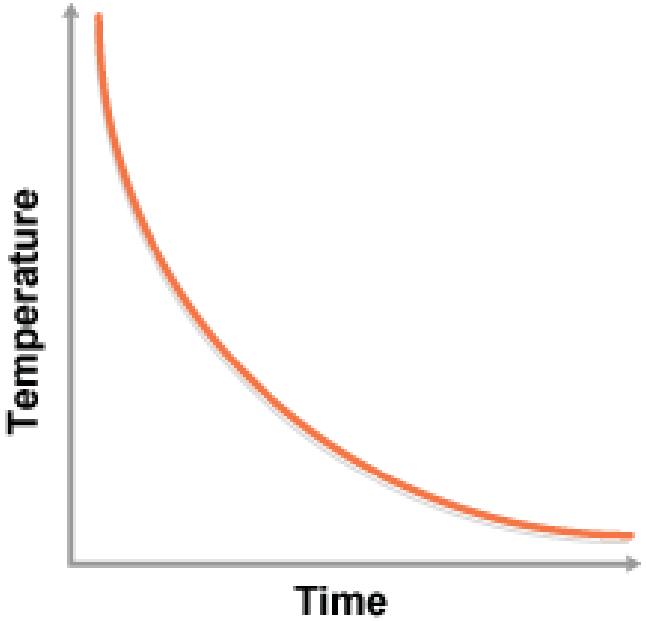
Simulated Annealing (SA)

- A hill-climbing algorithm that *never* goes downhill will get stuck in local maxima
- Better to do a random walk!
  - Will *eventually* find the global maximum
- Can we combine the two?
  - Make some downhill moves
  - But not completely randomly

Annealing

- In **metallurgy**, a technique of heating and slowly cooling metals/glass to reduce their defects and make them easier to machine





Physical Interpretation of Simulated Annealing

- simulated annealing:
  - free variables are like molecules
  - seek “low energy” (high quality) configuration
    - Note: minimization but equally applies to maximization
  - get this by slowly reducing temperature T, which controls how randomly particles move

Simulated Annealing

```
function SIMULATED-ANNEALING( problem, schedule) return a solution state
    input: problem, a problem
           schedule, a mapping from time to temperature
    local variables: current, a node.
                     next, a node.
                     T, a "temperature" controlling the probability of downward steps

    current ← MAKE-NODE(INITIAL-STATE[problem])
    for t ← 1 to ∞ do
        T ← schedule[t] # T -= 0.01
        if T = 0 then return current
        next ← a randomly selected successor of current
        ΔE ←  VALUE[next] - VALUE[current]
        if ΔE > 0 then current ← next
        else current ← next only with probability e^(ΔE/T)
```

Algorithm sketch: At each iteration, SA considers some neighbor, *next* of the current state *current*, and *probabilistically* decides whether to move from *next* to *current*.

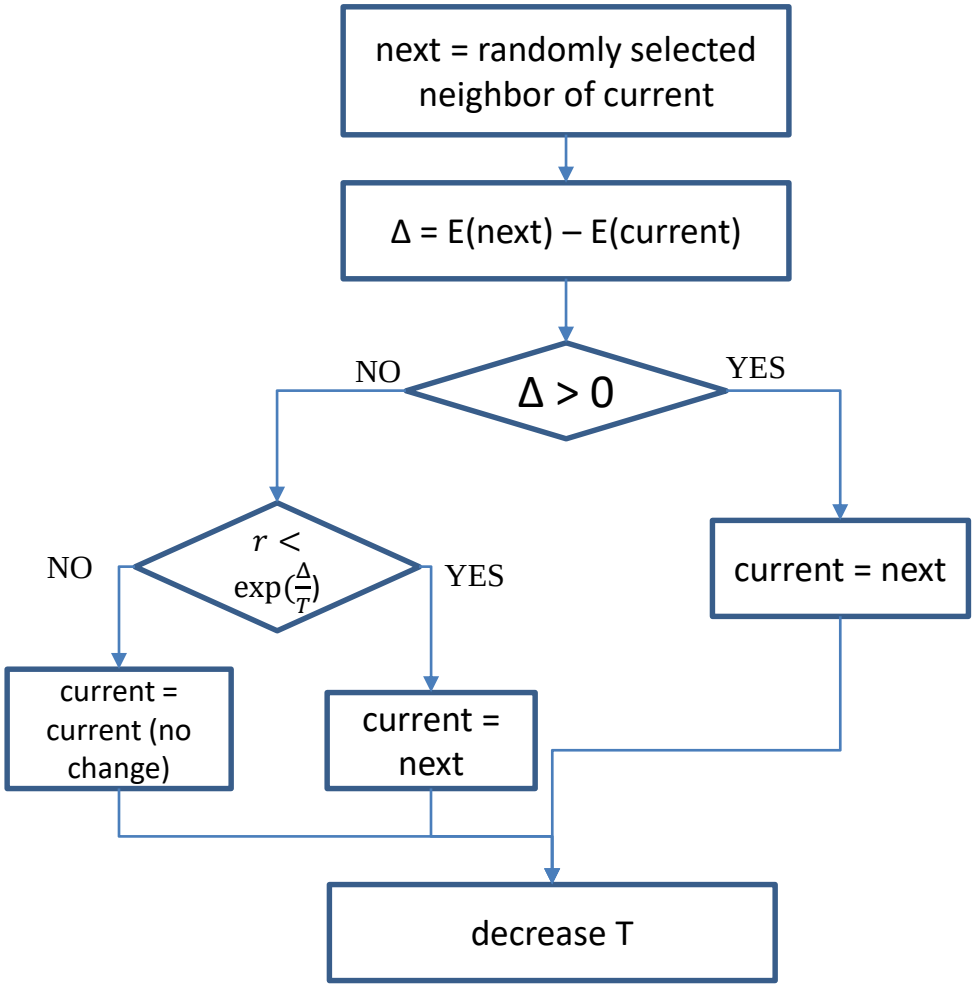
Repeat for a large number of iterations.

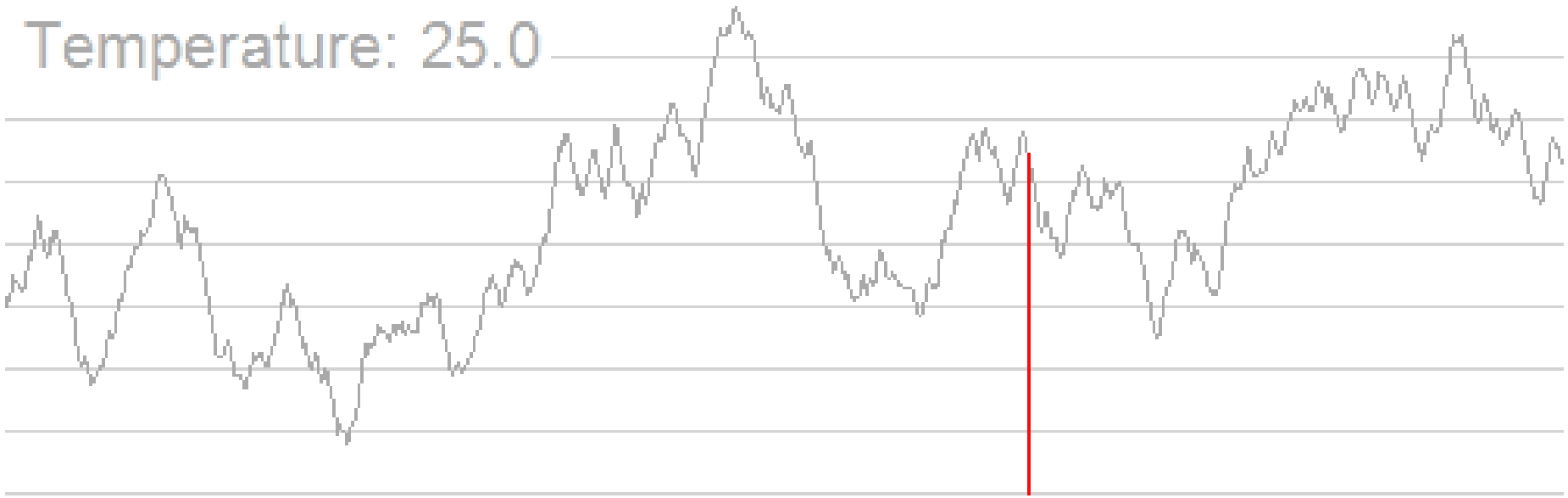
Search using Simulated Annealing

- Simulated Annealing = hill-climbing with **stochastic** moves

Basic ideas:

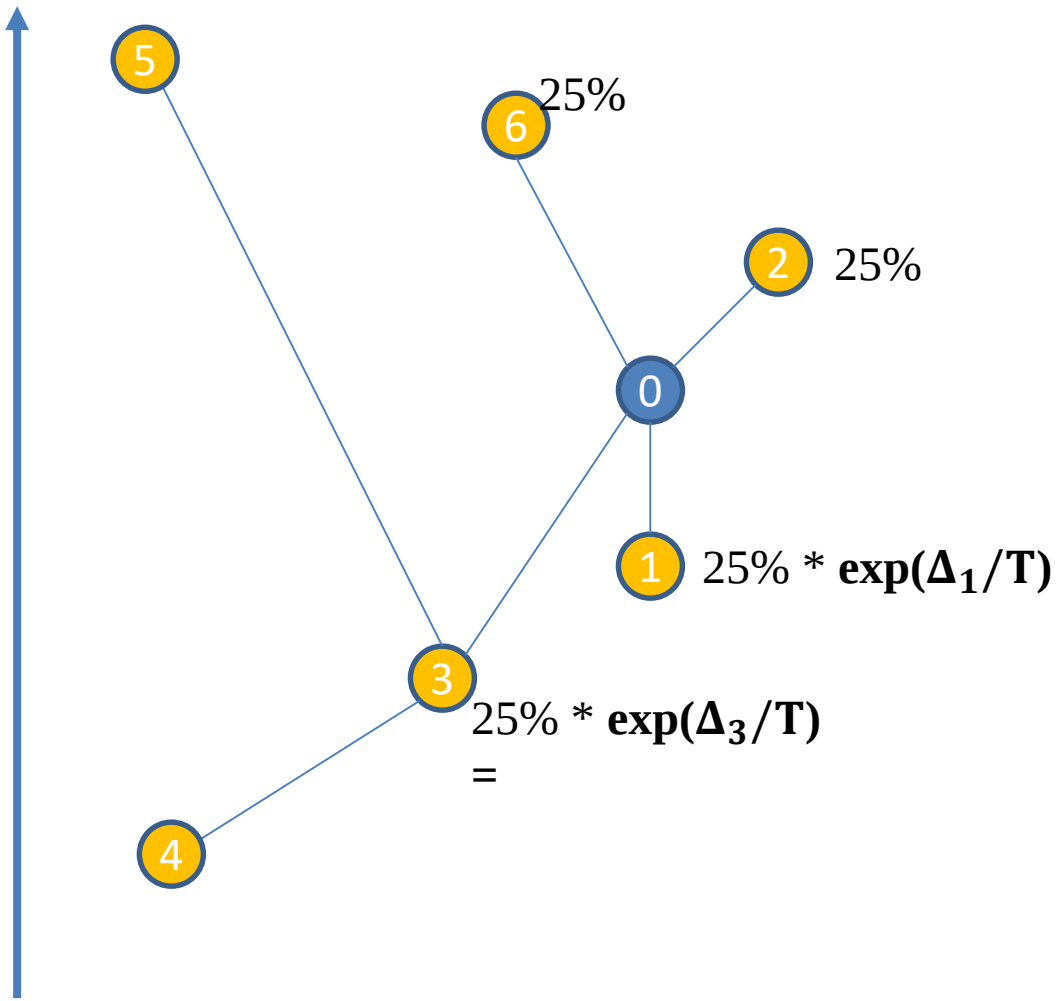
- like hill-climbing but instead of picking the best move, pick one **randomly**
- Δ = change in objective function value
- if Δ is positive, then move to that state
- Otherwise:
  - move to this state with **probability proportional to Δ**
  - thus: worse moves (large negative Δ) are executed less often
- There is always **a chance of escaping** from local maxima over time




Temperature: 25.0

Simulated Annealing example

- Current state value = 20
- Current temperature T = 1
- 3 moves/successors:
  - Objective function value = 19.1, ΔE_1 = -0.9
  - Value = 20.5, ΔE_2 = +0.5
  - Value = 15, ΔE_3 = -5
- pick a move randomly (with equal probability 1/3 = 0.333):
  - if E_2 is picked, move there (since $\Delta E_2 > 0$)
    - Prob(move to E_2) = 0.333
  - if E_1 or E_3 is picked, move there with probability = exp($\Delta E/T$)
  - move 1: prob. = exp(-0.9) = 0.406,
    - Prob(move to E_1) = 0.333 * 0.406 = 0.135
  - move 3: prob. = exp(-5) = 0.006
    - Prob(move to E_1) = 0.333 * 0.006 = 0.002


5
6
25%
2
25%
0
1
25% * $\mathbf{exp(\Delta_1/T)}$
3
25% * $\mathbf{exp(\Delta_3/T)}$
=
4

Class work

- Let current state value be 25
- Has 4 neighbors with values: 23, 20, 28, 30
- Calculate the probability of
  - Moving to 23
  - Moving to 20
  - Moving to 28
  - Moving to 30
  - Staying at 25
- Repeat for T=1 and T=10

Performance of Simulated Annealing

- T = "temperature" parameter
  - high T => probability of "locally bad" move is higher
  - low T => probability of "locally bad" move is lower
- T is decreased as the algorithm runs longer
  - *Annealing schedule*
- If the schedule is slow enough, SA finds the global optimum (with probability approaching 1)
- SA was proposed in 1983 by IBM researchers for solving VLSI layout problems (Kirkpatrick et al., Science, 220:671-680, 1983)
  - Also used for other problems: scheduling …
- Useful for some problems, but can be very **slow**
    - slowness comes about because T must be decreased very gradually to retain optimality
  - In practice **how do we decide the rate at which to decrease T?**
    - a practical problem with this method

Local search

GENETIC ALGORITHMS

Evolution by Natural Selection

- **Genome and heredity**
  - **Genome** is a complete set of chromosomes for a specie. Half of the genome (DNA) is passed from parent to child. Thus, heredity is passed through the genome.
- **Genetic variation in a population is**
  - the results of principal forces: **mutation** (various reasons) and **crossover** (sexual recombination) of genetic code (DNA)
- **Evolution is**
  - the **results** of **natural selection** (or **survival of the fittest**) over **multiple generations** through **Inheritance** (from ***reproduction***) and **genetic variation** (from ***crossover*** and ***mutation***).

What do these DNA and Evolution have to do with our problem solving approaches?

How to use Biological Knowledge in Problem Solving?

- **What we have learned from Biologists:**
  - How the information about functions and traits for an individual is **represented** and where it is **stored**.
  - **Evolutionary mechanism**
    - How is a member in a population evaluated: **Natural selection**
    - How does a population evolve? **Genetic variation**
    - What causes genetic variations?
      1. Inheritance
      2. Sexual reproduction: crossover of genes from both parents
      3. Mutation: random change
  - **Importance of diversity** in a population in evolution
    - History teaches us that all closed societies failed to survive.
- How can we use this knowledge to solve problems?
  - **Simulation** of the evolutionary mechanism
    - Genetic algorithms
    - Introduced by Holland and colleagues in 1975

GA: terminology

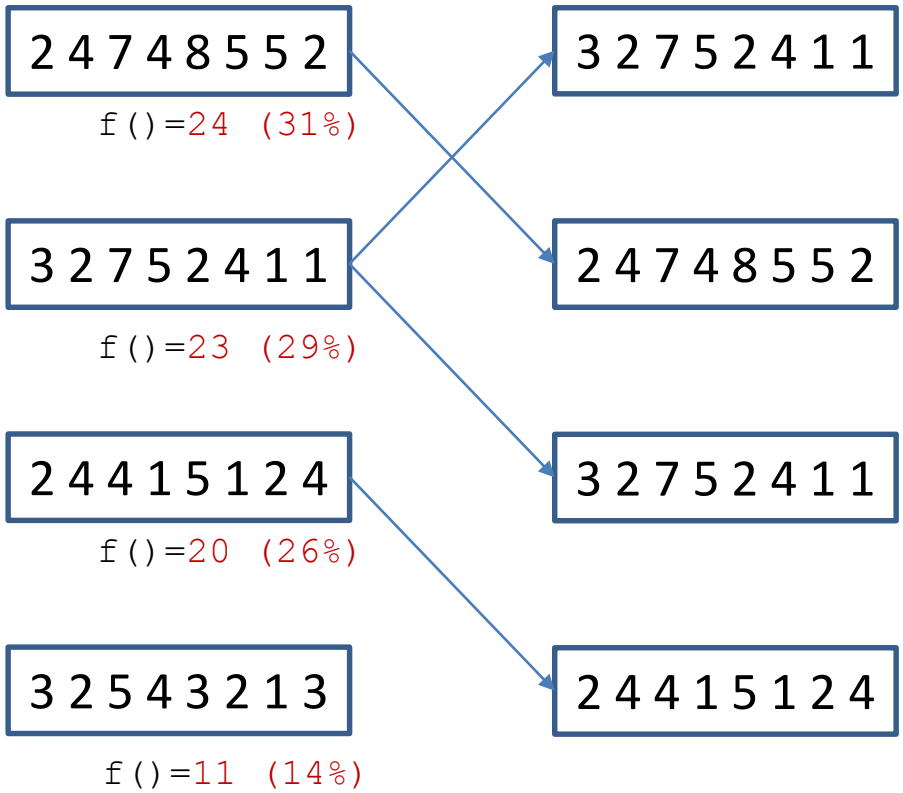
- **Population**
  - A set of candidate solutions/states
- **Crossover**
  - Combining parts of two states to create a child state
- **Mutation**
  - A random change in one bit of a state
- **Fitness function**
  - A numeric measure of the relative goodness of a state

Genetic Algorithm

```
function GENETIC-ALGORITHM(population, fitness) returns an individual
  repeat
      weights ←WEIGHTED-BY(population, fitness)
      population2 ← []
      for i = 1 to SIZE(population) do
          parent1 , parent2 ←WEIGHTED-RANDOM-CHOICES(population,weights, 2)
          child ←REPRODUCE(parent1, parent2 )  // crossover
          if (small random probability) then child ←MUTATE(child)
          add child to population2
      population ←population2
  until some individual is fit enough, or enough time has elapsed
  return the best individual in population, according to fitness
```

```
function REPRODUCE(parent1, parent2 ) returns an individual
      n←LENGTH(parent1)
      c←random number from 1 to n
      return APPEND(SUBSTRING(parent1, 1, c), SUBSTRING(parent2, c + 1, n))
```

8-queens problem

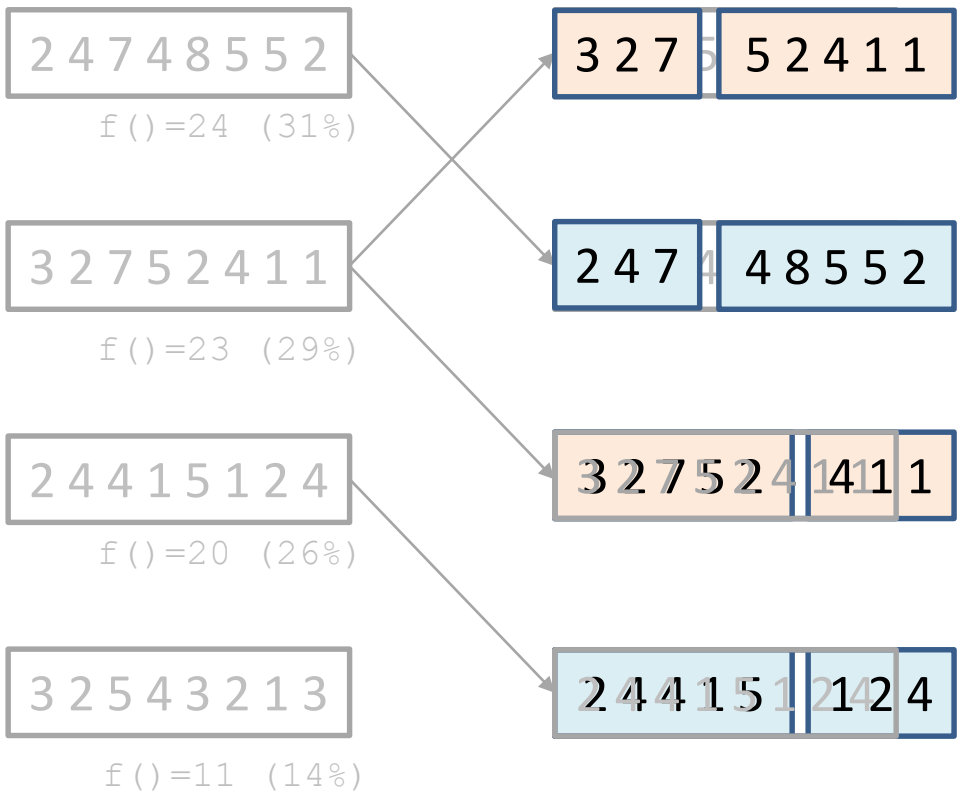


(a) Initial population

(b) Fitness function

(c) Selection

8-queens problem



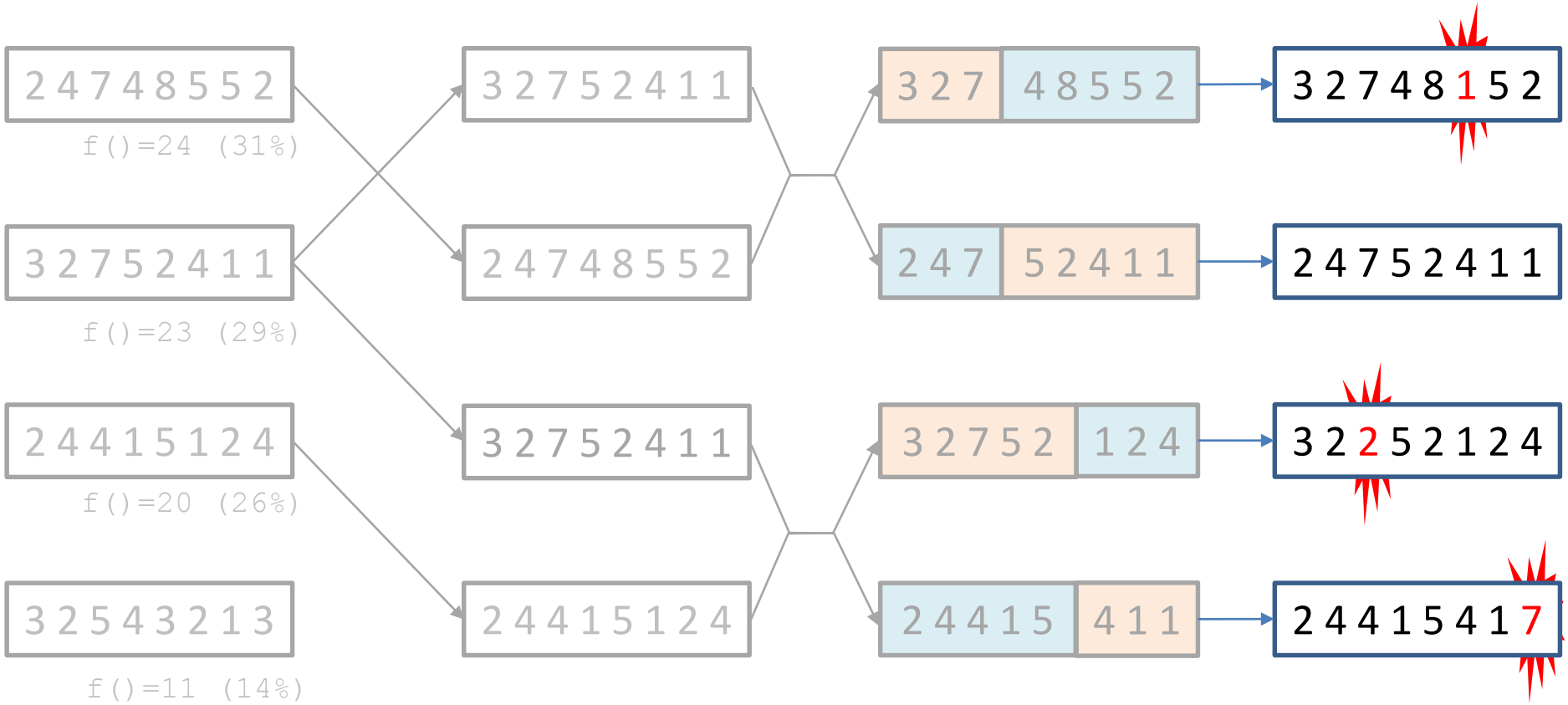
(a) Initial population

(b) Fitness function

(c) Selection

(d) Crossover

8-queens problem



(a) Initial population

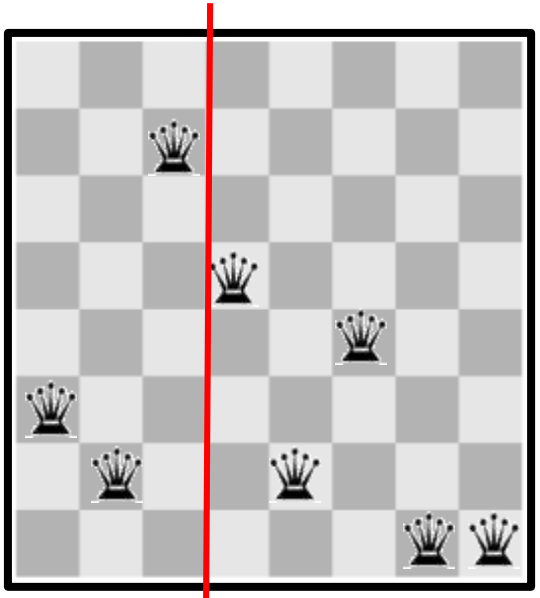
(b) Fitness function

(c) Selection

(d) Crossover

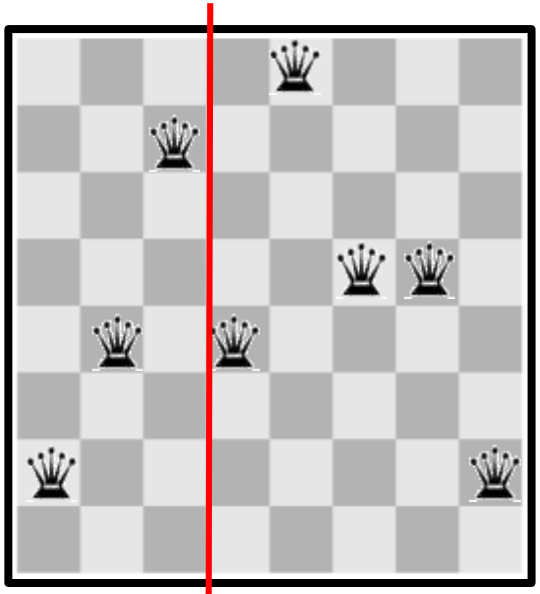
(e) Mutation

8-queens problem: crossover



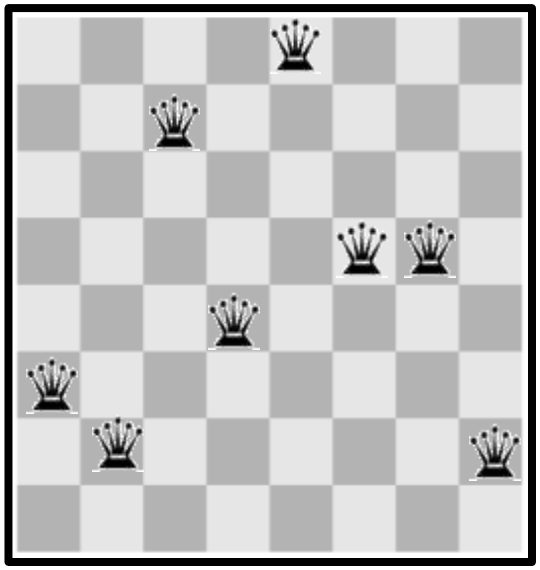
[3 2 7 | 5 2 4 1 1]

+



[2 4 7 | 4 8 5 5 2]

=



= [3 2 7 4 8 5 5 2]

Classwork: crossover

- Show a few other "children" of these two parent states
  - **[3 2 7 5 2 4 1 1]**
  - **[2 4 7 4 8 5 5 2]**

Setting up a GA for a specific problem

- Representation for a candidate solution (DNA)
  - Use a **fixed length string** (binary bit string).
- Initial population of candidate solutions (chromosomes)
  - Create randomly (each candidate solution can be a random binary string.)
- Fitness criteria (measure for survival of fitness)
  - Define a fitness function f(x) to evaluate each candidate solution.
- Creating next population with genetic variations (evolution)
  - Select survived candidate solutions.
  - Apply genetic operators to the selected solutions to produce improved solutions in next generation (offsprings with modified DNA).
- Terminating conditions and other parameters
  - population size, probability of crossover, probability of mutation, number of generations, etc.

Performance of GA

- Have been used in many optimization problems
  - Scheduling
  - Traveling Salesman Problem
- Works well for problems where *parts of a good candidate* should be left unchanged
- Disadvantages
  - Representation is not obvious and impacts how well it does
  - Several hyper-parameters:
    - Size of population, mutation probability, number of iterations, terminating condition, …

References

- George F. Luger, Artificial Intelligence: Structures and Strategies for Complex Problem Solving, 6th edition, Addison Wesley, 2009. **Chapters 4.1.1, 12.1**.
- Russel and Norvig, Artificial Intelligence: A Modern Approach, 3rd edition, Prentice Hall, 2010. **Chapter 4-4.1**